THE COMPLETE GUIDE

SQL Mastery

Beginner to Advanced

Concepts, code examples and interview questions with answers for every single topic.

15	101+	3
Topics	Interview Q&A	Levels

BEGINNER INTERMEDIATE ADVANCED

Save it. Practise daily. Crack the interview.

Your SQL Roadmap

Follow the topics in order. Each one has a short explanation, a code example, a pro tip and interview questions with model answers. Finish with the bonus problem set and the cheat sheet.

#	Topic	Level	Q&A
01	**SQL and Database Basics**	Beginner	6
02	**Creating Tables, Data Types and Constraints**	Beginner	6
03	**CRUD: INSERT, SELECT, UPDATE, DELETE**	Beginner	5
04	**Filtering and Sorting Data**	Beginner	6
05	**Aggregate Functions, GROUP BY and HAVING**	Beginner	6
06	**Joins**	Intermediate	6
07	**Subqueries and Set Operations**	Intermediate	6
08	**Functions, CASE and NULL Handling**	Intermediate	6
09	**Common Table Expressions (CTEs)**	Intermediate	6
10	**Window Functions**	Advanced	7
11	**Indexes and Query Optimization**	Advanced	7
12	**Views, Stored Procedures, Functions and Triggers**	Advanced	6
13	**Transactions, ACID and Concurrency**	Advanced	6
14	**Normalization and Schema Design**	Advanced	6
15	**Logical Query Order and Advanced Patterns**	Advanced	6
16	**Bonus: 10 must-practise SQL problems**	All	10
17	**Cheat sheet and interview strategy**	All	-

SAMPLE TABLES USED IN THIS GUIDE

employees

emp_id	name	dept_id	manager_id	salary
1	Asha	10	NULL	9000
2	Bilal	10	1	6500
3	Carla	20	1	7000
4	Dev	20	3	5000
5	Esha	NULL	3	4000

departments

dept_id	dept_name
10	Engineering
20	Sales
30	HR

Note: Esha has no department, and HR has no employees. This is perfect for practising joins and NULL handling.

01 SQL and Database Basics

BEGINNER

SQL (Structured Query Language) is the standard language for talking to relational databases such as MySQL, PostgreSQL, SQL Server, Oracle and SQLite. Data lives in **tables** made of rows (records) and columns (attributes). You describe *what* data you want and the database engine decides *how* to get it.

KEY POINTS

- **DDL** (structure): CREATE, ALTER, DROP, TRUNCATE
- **DML** (data): INSERT, UPDATE, DELETE
- **DQL** (querying): SELECT
- **DCL** (permissions): GRANT, REVOKE
- **TCL** (transactions): COMMIT, ROLLBACK, SAVEPOINT
- **Keys**: Primary, Foreign, Unique, Composite, Candidate, Surrogate

SQL | Example

```sql
-- Your first query
SELECT name, salary
FROM   employees
WHERE  dept_id = 10;
```

PRO TIP: Learn on a real engine. PostgreSQL or MySQL are free, and the skills transfer to every other database.

INTERVIEW QUESTIONS 6 questions with answers

Q1. What is SQL? Is it a programming language?

SQL is a **declarative** query language for relational databases. You state the result you want, not the steps. It is not a general-purpose language, but vendor extensions such as PL/SQL (Oracle) and T-SQL (SQL Server) add loops, variables and control flow.

Q2. What is the difference between DBMS and RDBMS?

A DBMS stores data in any structure (files, hierarchies). An RDBMS stores data in **related tables**, enforces keys and constraints, and supports ACID transactions. MySQL, PostgreSQL and Oracle are RDBMSs.

Q3. DELETE vs TRUNCATE vs DROP?

`DELETE` is DML: removes chosen rows (supports WHERE), is logged row by row, fires triggers. `TRUNCATE` is DDL: removes all rows quickly, no WHERE, usually resets auto-increment. `DROP` removes the whole table (structure and data).

Q4. What is the difference between a primary key and a unique key?

A table has only **one** primary key; it cannot be NULL and identifies each row. A table can have **many** unique keys, and they can allow NULL (how many NULLs depends on the database).

Q5. What is a foreign key?

A column (or set of columns) that references the primary key of another table. It enforces **referential integrity**, so you cannot insert an order for a customer that does not exist.

Q6. Name the different types of keys.

Super key (any set of columns that identifies a row), candidate key (minimal super key), primary key (the chosen candidate), alternate key (unchosen candidates), composite key (more than one column), foreign key, and surrogate key (artificial id).

02 Creating Tables, Data Types and Constraints

BEGINNER

DDL statements define the structure of your database. Choosing the right data type and constraints is what keeps data clean and queries fast.

KEY POINTS

- **Numbers**: INT, BIGINT, DECIMAL(p,s), FLOAT
- **Text**: CHAR(n), VARCHAR(n), TEXT
- **Date and time**: DATE, TIME, TIMESTAMP
- **Other**: BOOLEAN, JSON, UUID (varies by database)
- **Constraints**: NOT NULL, UNIQUE, PRIMARY KEY, FOREIGN KEY, CHECK, DEFAULT

SQL | DDL example

```sql
CREATE TABLE departments (
    dept_id    INT PRIMARY KEY,
    dept_name  VARCHAR(50) NOT NULL UNIQUE
);

CREATE TABLE employees (
    emp_id     INT PRIMARY KEY,
    name       VARCHAR(100) NOT NULL,
    dept_id    INT REFERENCES departments(dept_id) ON DELETE SET NULL,
    manager_id INT REFERENCES employees(emp_id),
    salary     DECIMAL(10,2) CHECK (salary > 0),
    hire_date  DATE DEFAULT CURRENT_DATE
);

ALTER TABLE employees ADD COLUMN email VARCHAR(100);
ALTER TABLE employees DROP COLUMN email;
```

PRO TIP: Always store money as DECIMAL, never FLOAT. Floats are approximate and create rounding errors.

INTERVIEW QUESTIONS 6 questions with answers

Q1. CHAR vs VARCHAR?

`CHAR(n)` is fixed length and pads with spaces; good for codes of constant size such as country codes. `VARCHAR(n)` is variable length and stores only what is needed; better for names and emails.

Q2. Why use DECIMAL instead of FLOAT for money?

DECIMAL stores exact values. FLOAT is binary and approximate, so `0.1 + 0.2` may not equal `0.3`. Financial data must be exact.

Q3. What happens if you delete a parent row that is referenced by a foreign key?

By default the delete is rejected. You can change the behaviour with `ON DELETE CASCADE` (delete children), `SET NULL`, `SET DEFAULT` or `RESTRICT / NO ACTION`.

Q4. Can a table have more than one primary key?

No, only one primary key per table. But that key can be **composite** (made of several columns), for example `(student_id, course_id)`.

Q5. What does the CHECK constraint do?

It validates a condition on every insert or update, for example `CHECK (salary > 0)`. Rows that fail are rejected.

Q6. What is the difference between a constraint and an index?

A constraint enforces a **rule** on the data. An index is a **performance structure**. They overlap because PRIMARY KEY and UNIQUE automatically create an index to enforce uniqueness.

03 CRUD: INSERT, SELECT, UPDATE, DELETE

BEGINNER

CRUD (Create, Read, Update, Delete) covers 80 percent of daily database work. Master these four statements first.

KEY POINTS

- `INSERT` adds rows (single or multi-row)
- `SELECT` reads rows
- `UPDATE` changes existing rows; always add WHERE
- `DELETE` removes rows; always add WHERE

SQL | CRUD in action

```sql
INSERT INTO departments (dept_id, dept_name)
VALUES (10, 'Engineering'), (20, 'Sales'), (30, 'HR');

INSERT INTO employees (emp_id, name, dept_id, salary)
VALUES (6, 'Farah', 10, 6000);

UPDATE employees SET salary = salary * 1.10 WHERE dept_id = 10;

DELETE FROM employees WHERE emp_id = 6;

SELECT * FROM employees;
```

PRO TIP: Before any UPDATE or DELETE, run the same WHERE as a SELECT first. Then wrap the change in a transaction.

INTERVIEW QUESTIONS 5 questions with answers

Q1. What happens if you run UPDATE or DELETE without a WHERE clause?

It affects **every row** in the table. Good habits: test with SELECT first, use a transaction so you can ROLLBACK, and enable safe-update mode in your client.

Q2. How do you insert multiple rows in one statement?

List several value tuples: `INSERT INTO t (a, b) VALUES (1,'x'), (2,'y'), (3,'z');`. It is faster than separate inserts.

Q3. What is INSERT ... SELECT used for?

It copies rows from a query into another table: `INSERT INTO archive SELECT * FROM orders WHERE order_date < '2023-01-01';`

Q4. What is an UPSERT?

Insert a row, or update it if it already exists. PostgreSQL: `INSERT ... ON CONFLICT DO UPDATE`. MySQL: `ON DUPLICATE KEY UPDATE`. SQL Server and Oracle: `MERGE`.

Q5. How do you create a copy of a table?

`CREATE TABLE new_t AS SELECT * FROM old_t;` (SQL Server: `SELECT * INTO new_t FROM old_t`). Add `WHERE 1 = 0` to copy only the structure.

04 Filtering and Sorting Data

BEGINNER

Use WHERE to filter rows, ORDER BY to sort them, and DISTINCT or LIMIT to shape the result.

KEY POINTS

- Comparison: `= <> < > <= >=`
- Logic: `AND OR NOT` (use brackets when mixing)
- Sets and ranges: `IN`, `BETWEEN` (inclusive), `LIKE` (`%` many chars, `_` one char)
- Missing data: `IS NULL`, `IS NOT NULL`
- Limit rows: `LIMIT n` (MySQL, PostgreSQL), `TOP n` (SQL Server), `FETCH FIRST n ROWS ONLY` (Oracle, standard)

SQL | Filtering and sorting

```sql
SELECT DISTINCT dept_id
FROM   employees
WHERE  dept_id IS NOT NULL;

SELECT name, salary
FROM   employees
WHERE  salary BETWEEN 5000 AND 8000
  AND  name LIKE 'A%'
ORDER BY salary DESC, name ASC
LIMIT  5;   -- SQL Server: SELECT TOP 5 ...
```

PRO TIP: NULL is not a value, it means unknown. Any comparison with NULL (even NULL = NULL) is unknown, never true.

INTERVIEW QUESTIONS 6 questions with answers

Q1. Why does `WHERE col = NULL` return nothing?

Because NULL means unknown, and any comparison with it evaluates to UNKNOWN (not TRUE). Use `IS NULL` or `IS NOT NULL`. SQL uses three-valued logic: TRUE, FALSE, UNKNOWN.

Q2. Is BETWEEN inclusive or exclusive?

Inclusive on both ends. `BETWEEN 5 AND 10` includes 5 and 10. Be careful with timestamps: `BETWEEN '2024-01-01' AND '2024-01-31'` misses most of Jan 31 if times are stored.

Q3. What do the LIKE wildcards % and _ do?

`%` matches zero or more characters, `_` matches exactly one. `'A%'` starts with A, `'_a%'` has 'a' as the second letter.

Q4. How do you get the 5 highest paid employees?

`ORDER BY salary DESC LIMIT 5` (MySQL/PostgreSQL), `SELECT TOP 5 ...` (SQL Server), `FETCH FIRST 5 ROWS ONLY` (Oracle/standard). If ties matter, use `DENSE_RANK()` instead.

Q5. What is the NOT IN trap with NULL?

If the subquery list contains even one NULL, `x NOT IN (...)` returns no rows, because the comparison becomes UNKNOWN. Prefer `NOT EXISTS` or filter NULLs out of the subquery.

Q6. What does DISTINCT do and what is its cost?

It removes duplicate rows from the result, applied across **all** selected columns. It needs sorting or hashing, so it can be expensive on large results.

05 Aggregate Functions, GROUP BY and HAVING

BEGINNER

Aggregates collapse many rows into one value. GROUP BY creates one result row per group, and HAVING filters those groups after aggregation.

KEY POINTS

- `COUNT`, `SUM`, `AVG`, `MIN`, `MAX`
- `COUNT(*)` counts rows; `COUNT(col)` counts non-NULL values; `COUNT(DISTINCT col)` counts unique non-NULL values
- Every non-aggregated column in SELECT must appear in GROUP BY
- `WHERE` filters rows **before** grouping; `HAVING` filters groups **after**

SQL | Grouping example

```sql
SELECT   dept_id,
         COUNT(*)    AS headcount,
         AVG(salary) AS avg_salary,
         MAX(salary) AS top_pay
FROM     employees
WHERE    dept_id IS NOT NULL
GROUP BY dept_id
HAVING   COUNT(*) >= 2
ORDER BY avg_salary DESC;
```

PRO TIP: Aggregate functions ignore NULLs (except COUNT(*)). AVG of 10, 20, NULL is 15, not 10.

INTERVIEW QUESTIONS 6 questions with answers

Q1. WHERE vs HAVING?

`WHERE` filters individual rows before grouping and cannot use aggregates. `HAVING` filters groups after aggregation and can use aggregates, e.g. `HAVING COUNT(*) > 5`.

Q2. COUNT(*) vs COUNT(column) vs COUNT(DISTINCT column)?

`COUNT(*)` counts all rows including NULLs. `COUNT(col)` skips NULLs in that column. `COUNT(DISTINCT col)` counts unique non-NULL values.

Q3. Why do you get an error when selecting a column that is not in GROUP BY?

Because that column has many values per group and the database does not know which one to show. Add it to GROUP BY or wrap it in an aggregate. (MySQL with certain modes may allow it, but results are unpredictable.)

Q4. How do you find duplicate records?

```sql
SELECT email, COUNT(*) FROM users GROUP BY email HAVING COUNT(*) > 1;
```

Q5. How do aggregate functions treat NULL?

They ignore NULL values, except `COUNT(*)`. If all values are NULL, SUM and AVG return NULL, while COUNT(col) returns 0.

Q6. Can you use a column alias in WHERE?

No. WHERE runs before SELECT, so the alias does not exist yet. You can use it in ORDER BY. Repeat the expression or use a subquery or CTE.

06 Joins

INTERMEDIATE

Joins combine rows from two or more tables based on a related column. They are the most tested topic in SQL interviews.

KEY POINTS

- **INNER JOIN**: only rows that match in both tables
- **LEFT JOIN**: all rows from the left table, matches from the right (NULL if none)
- **RIGHT JOIN**: mirror of LEFT JOIN
- **FULL OUTER JOIN**: all rows from both sides
- **CROSS JOIN**: every combination (Cartesian product)
- **SELF JOIN**: a table joined with itself, e.g. employee and manager

SQL | Join patterns

```sql
-- INNER: employees that have a department
SELECT e.name, d.dept_name
FROM   employees e
JOIN   departments d ON e.dept_id = d.dept_id;

-- Anti-join: departments with NO employees
SELECT d.dept_name
FROM   departments d
LEFT JOIN employees e ON e.dept_id = d.dept_id
WHERE  e.emp_id IS NULL;

-- Self join: employee and their manager
SELECT e.name AS employee, m.name AS manager
FROM   employees e
LEFT JOIN employees m ON e.manager_id = m.emp_id;
```

PRO TIP: To find 'rows with no match', use LEFT JOIN plus WHERE right_table.key IS NULL.

INTERVIEW QUESTIONS 6 questions with answers

Q1. Explain INNER JOIN vs LEFT JOIN.

INNER JOIN returns only matched rows. LEFT JOIN returns **all** rows from the left table, and fills right-side columns with NULL when there is no match.

Q2. How do you find customers who have never placed an order?

`SELECT c.name FROM customers c LEFT JOIN orders o ON o.customer_id = c.id WHERE o.id IS NULL;` (or use `NOT EXISTS`).

Q3. What is a self join and when do you use it?

Joining a table to itself using aliases. Typical uses: employee-manager hierarchies, comparing rows within the same table (e.g. employees earning more than their manager).

Q4. What is the difference between a filter in ON and in WHERE for a LEFT JOIN?

A condition on the right table in `ON` keeps unmatched left rows (right columns become NULL). The same condition in `WHERE` removes those NULL rows and silently turns the LEFT JOIN into an INNER JOIN.

Q5. What is a CROSS JOIN?

It returns the Cartesian product: every row of A paired with every row of B (m x n rows). Useful for generating combinations, such as every size with every colour.

Q6. Table A has 5 rows and table B has 4. How many rows can each join return?

CROSS JOIN: exactly 20. INNER JOIN: from 0 up to 20 (duplicate keys can push it above the smaller table). LEFT JOIN: at least 5. FULL JOIN: at least 5, and 9 when nothing matches (more if keys repeat).

07 Subqueries and Set Operations

INTERMEDIATE

A subquery is a query inside another query. Set operations stack the results of two queries on top of each other.

KEY POINTS

- **Scalar subquery**: returns one value
- **Table subquery**: returns rows, used with IN, EXISTS or in FROM
- **Correlated subquery**: refers to the outer query, evaluated per outer row
- `UNION` (removes duplicates), `UNION ALL` (keeps duplicates, faster)
- `INTERSECT` (common rows), `EXCEPT` / `MINUS` (rows in first but not second)

SQL | Subquery patterns

```sql
-- Scalar: above company average
SELECT name FROM employees
WHERE  salary > (SELECT AVG(salary) FROM employees);

-- Correlated: above their OWN department average
SELECT e.name FROM employees e
WHERE  e.salary > (SELECT AVG(salary) FROM employees
                   WHERE dept_id = e.dept_id);

-- EXISTS: departments that have employees
SELECT d.dept_name FROM departments d
WHERE  EXISTS (SELECT 1 FROM employees e
               WHERE e.dept_id = d.dept_id);
```

PRO TIP: UNION ALL is faster than UNION because it skips duplicate removal. Use it whenever duplicates are impossible or acceptable.

INTERVIEW QUESTIONS 6 questions with answers

Q1. UNION vs UNION ALL?

`UNION` removes duplicate rows (extra sort or hash step). `UNION ALL` keeps everything and is faster. Both need the same number of columns with compatible data types.

Q2. What is a correlated subquery?

A subquery that references columns from the outer query, so it is logically re-evaluated for each outer row. It can be slow; often a join or window function does the same job faster.

Q3. IN vs EXISTS?

`EXISTS` stops at the first match and is NULL-safe. `IN` compares against a list and misbehaves with NULLs when negated (`NOT IN`). Modern optimizers often plan them identically, but `EXISTS` is the safer habit.

Q4. Subquery or JOIN: which is faster?

It depends, and optimizers often rewrite one into the other. Use JOIN when you need columns from both tables. Use EXISTS or IN when you only need to test for presence. Always check the execution plan.

Q5. Find employees who earn more than the average salary of their department.

Use the correlated subquery above, or better: join to a pre-aggregated CTE, or compare against `AVG(salary) OVER (PARTITION BY dept_id)`.

Q6. What is the difference between INTERSECT and INNER JOIN?

INTERSECT compares whole rows across two result sets and removes duplicates. INNER JOIN matches on specific key columns and can return columns from both tables and duplicate rows.

08 Functions, CASE and NULL Handling

INTERMEDIATE

Built-in functions transform data inside the query. CASE adds if/else logic, and COALESCE and NULLIF tame NULLs.

KEY POINTS

- **String**: CONCAT, UPPER, LOWER, TRIM, SUBSTRING, LENGTH, REPLACE
- **Numeric**: ROUND, CEIL, FLOOR, ABS, MOD
- **Date**: CURRENT_DATE, EXTRACT, DATE_TRUNC (PostgreSQL), DATEDIFF (MySQL, SQL Server)
- **NULL**: `COALESCE(a, b, c)` returns the first non-NULL; `NULLIF(a, b)` returns NULL when a = b
- **CASE**: conditional logic anywhere (SELECT, ORDER BY, GROUP BY, aggregates)

SQL | Functions and CASE

```sql
SELECT name,
       UPPER(name)                    AS name_upper,
       COALESCE(dept_id, 0)           AS dept_or_zero,
       CASE WHEN salary >= 8000 THEN 'High'
            WHEN salary >= 5000 THEN 'Mid'
            ELSE 'Low' END            AS band
FROM   employees;

-- Conditional aggregation (a mini pivot)
SELECT COUNT(CASE WHEN salary >= 7000 THEN 1 END) AS high_earners,
       COUNT(CASE WHEN salary <  7000 THEN 1 END) AS others
FROM   employees;

-- Safe division
SELECT total / NULLIF(count_rows, 0) FROM stats;
```

PRO TIP: CASE evaluates top to bottom and stops at the first true condition. Order your WHEN branches carefully.

INTERVIEW QUESTIONS 6 questions with answers

Q1. COALESCE vs ISNULL vs IFNULL vs NVL?

`COALESCE` is ANSI standard, takes many arguments and works everywhere. `ISNULL` (SQL Server), `IFNULL` (MySQL) and `NVL` (Oracle) are vendor-specific two-argument versions.

Q2. How do you avoid a divide-by-zero error?

Wrap the divisor in `NULLIF(divisor, 0)`. The division then returns NULL instead of failing. Add `COALESCE` if you want a default.

Q3. How can CASE be used inside an aggregate?

Conditional aggregation: `SUM(CASE WHEN status = 'paid' THEN amount ELSE 0 END)`. This is the standard way to pivot rows into columns without a PIVOT keyword.

Q4. How would you calculate someone's age or the days between two dates?

It is database-specific: MySQL `DATEDIFF(d1, d2)`, SQL Server `DATEDIFF(day, d1, d2)`, PostgreSQL `d1 - d2` or `AGE(d1, d2)`. Interviewers want to see you know the dialect you work in.

Q5. Can CASE be used in ORDER BY?

Yes, for custom sort orders: `ORDER BY CASE status WHEN 'urgent' THEN 1 WHEN 'normal' THEN 2 ELSE 3 END`.

Q6. What does NULL + 5 return? What does CONCAT with NULL return?

`NULL + 5` is NULL. String concatenation with `||` or `+` usually gives NULL, but `CONCAT()` in MySQL and SQL Server skips or treats NULL as empty, depending on the database. Always check or use COALESCE.

09 Common Table Expressions (CTEs)

INTERMEDIATE

A CTE (WITH clause) is a named temporary result set that lives for one statement. It makes complex queries readable and enables recursion.

KEY POINTS

- Syntax: `WITH name AS (SELECT ...) SELECT ... FROM name`
- You can chain several CTEs, and later ones can use earlier ones
- A **recursive CTE** has an anchor member, a recursive member (UNION ALL) and a stopping condition
- Great for hierarchies, org charts, bill of materials and generating date series

SQL | CTE examples

```sql
WITH dept_avg AS (
    SELECT dept_id, AVG(salary) AS avg_sal
    FROM   employees
    GROUP BY dept_id
)
SELECT e.name, e.salary, d.avg_sal
FROM   employees e
JOIN   dept_avg d ON e.dept_id = d.dept_id
WHERE  e.salary > d.avg_sal;

-- Recursive CTE: org chart (SQL Server omits RECURSIVE)
WITH RECURSIVE org AS (
    SELECT emp_id, name, manager_id, 1 AS lvl
    FROM   employees WHERE manager_id IS NULL
  UNION ALL
    SELECT e.emp_id, e.name, e.manager_id, o.lvl + 1
    FROM   employees e JOIN org o ON e.manager_id = o.emp_id
)
SELECT * FROM org;
```

PRO TIP: Think of a CTE as a readable variable name for a subquery. If you reuse the logic several times, a CTE keeps it in one place.

INTERVIEW QUESTIONS 6 questions with answers

Q1. What is a CTE and why use it?

A named, temporary result set defined with `WITH`. It improves readability, lets you reuse a subquery multiple times in one statement, and supports recursion.

Q2. CTE vs subquery vs temp table vs view?

A subquery is inline and unnamed. A CTE is named, single-statement scope. A temp table is physically stored for the session and can be indexed. A view is a saved query stored in the database for reuse across statements.

Q3. Is a CTE materialized (stored) or just a macro?

Depends on the engine. SQL Server treats it like an inline view. PostgreSQL 12+ inlines it unless it is referenced more than once or marked `MATERIALIZED`. Never assume a CTE caches results.

Q4. How does a recursive CTE work?

It runs the anchor query once, then repeatedly runs the recursive member against the previous iteration's rows, appending results until no new rows are produced.

Q5. How do you prevent infinite recursion?

Make sure the recursion moves toward a base case, add a depth limit (`WHERE lvl < 20`), detect cycles (track visited ids), or use engine limits such as SQL Server `OPTION (MAXRECURSION n)`.

Q6. Give real-world uses of recursive CTEs.

Org chart and manager chains, category trees, bill of materials, graph and path traversal, and generating a calendar of dates.

10 Window Functions

ADVANCED

Window functions calculate across a set of related rows **without collapsing them** like GROUP BY does. They are the number one topic in advanced SQL interviews.

KEY POINTS

- Syntax: `function() OVER (PARTITION BY ... ORDER BY ... frame)`
- **Ranking**: ROW_NUMBER, RANK, DENSE_RANK, NTILE
- **Navigation**: LAG, LEAD, FIRST_VALUE, LAST_VALUE
- **Aggregates as windows**: SUM, AVG, COUNT, MIN, MAX with OVER
- Frame example: `ROWS BETWEEN 2 PRECEDING AND CURRENT ROW` for a 3-row moving average

SQL | Window function showcase

```sql
SELECT name, dept_id, salary,
  ROW_NUMBER() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rn,
  RANK()       OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rnk,
  DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS drnk,
  SUM(salary)  OVER (PARTITION BY dept_id ORDER BY emp_id)      AS running_total,
  LAG(salary)  OVER (ORDER BY emp_id)                           AS prev_salary,
  AVG(salary)  OVER (PARTITION BY dept_id)                      AS dept_avg
FROM employees;
```

PRO TIP: You cannot filter on a window function in WHERE. Compute it in a CTE or subquery, then filter outside.

INTERVIEW QUESTIONS 7 questions with answers

Q1. ROW_NUMBER vs RANK vs DENSE_RANK?

For scores 100, 90, 90, 80: `ROW_NUMBER` gives 1,2,3,4 (always unique). `RANK` gives 1,2,2,4 (ties share a rank, gaps follow). `DENSE_RANK` gives 1,2,2,3 (ties share a rank, no gaps).

Q2. Window function vs GROUP BY?

GROUP BY collapses rows into one per group. A window function keeps every row and adds the calculated value alongside, so you can show detail and aggregate together.

Q3. Find the 3rd highest salary in each department.

Rank inside a CTE: `DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS r`, then `WHERE r = 3`.

Q4. How do you compute month-over-month growth?

Use LAG: `revenue - LAG(revenue) OVER (ORDER BY month)` for absolute change, and divide by the previous value (protected with NULLIF) for percentage change.

Q5. Why can't window functions be used in WHERE?

Logical order: FROM, WHERE, GROUP BY, HAVING, then SELECT (where windows are computed). WHERE runs before windows exist. Wrap in a subquery or CTE. Snowflake and BigQuery offer `QUALIFY` for this.

Q6. What is the default window frame, and why does it matter?

With `ORDER BY`, the default is `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`. That makes SUM a running total, but `LAST_VALUE` returns the current row unless you extend the frame to `UNBOUNDED FOLLOWING`.

Q7. How do you calculate a 7-day moving average?

`AVG(sales) OVER (ORDER BY day ROWS BETWEEN 6 PRECEDING AND CURRENT ROW)`.

11 Indexes and Query Optimization

ADVANCED

An index is a sorted lookup structure (usually a B-tree) that lets the database find rows without scanning the whole table. Reads get faster, writes get slightly slower.

KEY POINTS

- **Clustered**: defines physical row order; one per table (InnoDB and SQL Server)
- **Non-clustered**: separate structure pointing to rows; many per table
- **Composite index** `(a, b, c)` follows the **leftmost prefix** rule
- **Covering index**: contains every column the query needs, so the table is never touched
- Read plans with `EXPLAIN` (or `EXPLAIN ANALYZE`)

SQL | Index and plan

```sql
CREATE INDEX idx_emp_dept_salary ON employees (dept_id, salary);

EXPLAIN ANALYZE
SELECT name FROM employees WHERE dept_id = 10 AND salary > 5000;

-- BAD: function on the column blocks the index
SELECT * FROM employees WHERE YEAR(hire_date) = 2024;

-- GOOD: sargable range predicate
SELECT * FROM employees
WHERE  hire_date >= '2024-01-01' AND hire_date < '2025-01-01';
```

PRO TIP: Index the columns you filter, join and sort on. Do not index everything: each index costs storage and slows INSERT/UPDATE.

INTERVIEW QUESTIONS 7 questions with answers

Q1. What is an index and what are its drawbacks?

A data structure that speeds up lookups, sorts and joins. Drawbacks: extra storage, slower INSERT, UPDATE and DELETE because indexes must be maintained, and too many indexes confuse the optimizer.

Q2. Clustered vs non-clustered index?

A clustered index stores the table rows in key order (so only one per table). A non-clustered index is a separate structure holding key values and pointers to the rows. A table without a clustered index is called a heap.

Q3. When will the database NOT use an index?

Function or calculation on the indexed column, leading wildcard (`LIKE '%abc'`), implicit type conversion, low-selectivity columns (e.g. boolean), `OR` across different columns, and tiny tables where a scan is cheaper.

Q4. Explain the leftmost prefix rule.

An index on `(a, b, c)` can serve filters on `a`, `a,b` and `a,b,c`, but not `b` alone or `c` alone.

Q5. What is a covering index?

An index that includes all columns referenced by the query, so the engine answers from the index alone (an index-only scan).

Q6. How would you optimize a slow query?

Read the execution plan, add or fix indexes, select only needed columns (no `SELECT *`), filter early, avoid functions on indexed columns, replace correlated subqueries with joins, update statistics, and use keyset pagination for deep pages.

Q7. Index scan vs index seek vs table scan?

A **seek** jumps straight to matching entries (best). An **index scan** reads the whole index. A **table scan** reads every row of the table.

12 Views, Stored Procedures, Functions and Triggers

ADVANCED

These are programmable objects stored inside the database. They encapsulate logic, improve security and reuse.

KEY POINTS

- **View**: a saved SELECT that behaves like a virtual table
- **Materialized view**: stores the result physically; must be refreshed
- **Stored procedure**: reusable block of SQL, can change data, takes IN/OUT parameters
- **Function (UDF)**: returns a value and can be used inside queries
- **Trigger**: runs automatically BEFORE or AFTER an INSERT, UPDATE or DELETE

SQL | Programmable objects

```sql
CREATE VIEW v_dept_summary AS
SELECT dept_id, COUNT(*) AS headcount, AVG(salary) AS avg_sal
FROM   employees GROUP BY dept_id;

-- Stored procedure (MySQL syntax)
DELIMITER //
CREATE PROCEDURE give_raise(IN p_dept INT, IN p_pct DECIMAL(5,2))
BEGIN
  UPDATE employees
  SET    salary = salary * (1 + p_pct / 100)
  WHERE  dept_id = p_dept;
END //
DELIMITER ;

-- Audit trigger (MySQL syntax)
CREATE TRIGGER trg_salary_audit AFTER UPDATE ON employees
FOR EACH ROW
  INSERT INTO salary_audit (emp_id, old_sal, new_sal)
  VALUES (OLD.emp_id, OLD.salary, NEW.salary);
```

PRO TIP: Syntax for procedures and triggers differs a lot between databases. Learn the concepts first, then your engine's dialect.

INTERVIEW QUESTIONS 6 questions with answers

Q1. View vs materialized view?

A view stores only the query and runs it every time. A materialized view stores the result set, so reads are fast, but data can be stale until refreshed.

Q2. Can you INSERT or UPDATE through a view?

Simple views on one table usually can. Views with joins, aggregates, DISTINCT or GROUP BY generally cannot be updated directly.

Q3. Stored procedure vs function?

A function must return a value and can be used inside SELECT. A procedure can return zero, one or many results, can modify data, manage transactions and is called with `CALL` or `EXEC`.

Q4. What is a trigger? What are the downsides?

Code that fires automatically on a table event. Useful for auditing and enforcing rules. Downsides: hidden logic, harder debugging, performance cost, and cascading triggers.

Q5. What is a cursor and why is it discouraged?

A cursor processes rows one at a time. It is slow and resource heavy compared with set-based SQL. Use it only when row-by-row logic is truly unavoidable.

Q6. Benefits of stored procedures?

Reusability, centralized business logic, less network traffic, precompiled plans, and tighter security (grant EXECUTE without table access).

13 Transactions, ACID and Concurrency

ADVANCED

A transaction groups several statements into one all-or-nothing unit of work. Concurrency control keeps simultaneous users from corrupting each other's data.

KEY POINTS

- **A**tomicity: all or nothing
- **C**onsistency: data moves from one valid state to another
- **I**solation: concurrent transactions do not interfere
- **D**urability: committed data survives crashes
- Commands: `BEGIN` / `START TRANSACTION`, `COMMIT`, `ROLLBACK`, `SAVEPOINT`

SQL | Transaction example

```sql
BEGIN;
UPDATE accounts SET balance = balance - 500 WHERE id = 1;
UPDATE accounts SET balance = balance + 500 WHERE id = 2;
COMMIT;            -- or ROLLBACK if anything failed

-- Pessimistic locking: lock the row until commit
SELECT * FROM accounts WHERE id = 1 FOR UPDATE;
```

PRO TIP: Keep transactions short. Long transactions hold locks, block other users and cause deadlocks.

INTERVIEW QUESTIONS 6 questions with answers

Q1. Explain ACID with a bank transfer example.

Transfer 500: both the debit and credit succeed or neither does (**atomicity**); balances never go negative if a CHECK forbids it (**consistency**); other sessions never see a half-finished transfer (**isolation**); once committed, a power cut cannot undo it (**durability**).

Q2. What are the isolation levels and the problems they prevent?

See the table below. Higher isolation means fewer anomalies but more locking and lower concurrency.

Q3. Dirty read vs non-repeatable read vs phantom read?

Dirty read: reading uncommitted data. **Non-repeatable read**: the same row returns different values within one transaction. **Phantom read**: the same query returns a different set of rows because another transaction inserted or deleted matching rows.

Q4. What is a deadlock and how do you avoid it?

Two transactions each hold a lock the other needs, so neither can proceed; the DB kills one (the victim). Avoid by accessing tables and rows in a consistent order, keeping transactions short, and using appropriate indexes and isolation levels.

Q5. Optimistic vs pessimistic locking?

Pessimistic locks the data up front (`SELECT ... FOR UPDATE`). Optimistic assumes conflicts are rare and checks a version number or timestamp at update time, retrying if it changed.

Q6. What is a SAVEPOINT?

A marker inside a transaction that you can roll back to without undoing the whole transaction: `SAVEPOINT s1; ... ROLLBACK TO s1;`.

ISOLATION LEVELS AT A GLANCE

Isolation level	Dirty read	Non-repeatable read	Phantom read
Read Uncommitted	Possible	Possible	Possible
Read Committed	Prevented	Possible	Possible
Repeatable Read	Prevented	Prevented	Possible*
Serializable	Prevented	Prevented	Prevented

*Depends on the database: PostgreSQL and MySQL InnoDB prevent most phantoms at Repeatable Read.

14 Normalization and Schema Design

ADVANCED

Normalization organizes tables to remove redundancy and update anomalies. Good design makes everything else, including queries, simpler and faster.

KEY POINTS

- **1NF**: atomic values, no repeating groups
- **2NF**: 1NF plus no partial dependency on part of a composite key
- **3NF**: 2NF plus no transitive dependency (non-key column depending on another non-key column)
- **BCNF**: stricter 3NF; every determinant is a candidate key
- **Denormalization**: deliberate redundancy for read speed (reporting, analytics)

SQL | Junction table

```sql
-- Many-to-many via a junction table
CREATE TABLE student_course (
    student_id  INT,
    course_id   INT,
    enrolled_on DATE,
    PRIMARY KEY (student_id, course_id),
    FOREIGN KEY (student_id) REFERENCES students(id),
    FOREIGN KEY (course_id)  REFERENCES courses(id)
);
```

PRO TIP: Normalize for OLTP (many writes). Denormalize carefully for OLAP (heavy reads and reporting).

INTERVIEW QUESTIONS 6 questions with answers

Q1. Explain 1NF, 2NF and 3NF with a simple example.

1NF: do not store 'Math, Physics' in one cell; one value per cell. **2NF**: in `(order_id, product_id, product_name)`, product_name depends only on product_id, so move it out. **3NF**: if `zip` determines `city`, store city in a zip table, not in every customer row.

Q2. When would you denormalize?

When read performance matters more than storage and update simplicity, such as dashboards, data warehouses and pre-computed reporting tables, accepting the cost of keeping copies in sync.

Q3. OLTP vs OLAP?

OLTP handles many short transactions (orders, payments) on normalized schemas. OLAP handles complex analytical queries on large historical data, usually with denormalized star schemas.

Q4. How do you model a many-to-many relationship?

With a **junction (bridge) table** holding the foreign keys of both sides, usually as a composite primary key.

Q5. Star schema vs snowflake schema?

A star schema has one central **fact** table (measures) surrounded by denormalized **dimension** tables. A snowflake normalizes the dimensions into sub-tables: less redundancy, more joins.

Q6. Surrogate key vs natural key?

A surrogate key is an artificial identifier (auto-increment, UUID) with no business meaning; stable and compact. A natural key is a real-world attribute (email, passport number) that can change or be reused.

15 Logical Query Order and Advanced Patterns

ADVANCED

Understanding how SQL is *logically* processed explains most errors and is a favourite interview question. Then practise the classic patterns below.

KEY POINTS

- **Logical order**: FROM / JOIN, WHERE, GROUP BY, HAVING, SELECT (incl. windows), DISTINCT, ORDER BY, LIMIT
- That is why aliases from SELECT work in ORDER BY but not in WHERE
- Patterns: dedupe with ROW_NUMBER, top-N per group, gaps and islands, running totals, keyset pagination

SQL | Advanced patterns

```sql
-- Delete duplicates, keep the lowest id (PostgreSQL / SQL Server)
WITH d AS (
  SELECT id, ROW_NUMBER() OVER (PARTITION BY email ORDER BY id) AS rn
  FROM users)
DELETE FROM users WHERE id IN (SELECT id FROM d WHERE rn > 1);

-- Top 2 salaries per department
SELECT * FROM (
  SELECT *, DENSE_RANK() OVER (PARTITION BY dept_id
                               ORDER BY salary DESC) AS rnk
  FROM employees) t
WHERE rnk <= 2;

-- Consecutive-day streaks (gaps and islands, PostgreSQL)
WITH x AS (
  SELECT DISTINCT user_id, login_date FROM logins),
g AS (
  SELECT user_id, login_date,
         login_date - (ROW_NUMBER() OVER (PARTITION BY user_id
                       ORDER BY login_date))::int AS grp
  FROM x)
SELECT user_id, MIN(login_date) AS start_d, MAX(login_date) AS end_d,
       COUNT(*) AS streak
FROM g GROUP BY user_id, grp;

-- Keyset pagination (fast at any depth)
SELECT * FROM orders WHERE id > 1000 ORDER BY id LIMIT 20;
```

PRO TIP: Gaps and islands trick: for consecutive values, value minus ROW_NUMBER() stays constant inside one streak.

INTERVIEW QUESTIONS 6 questions with answers

Q1. Explain the logical order of query execution.

FROM and JOINs, WHERE, GROUP BY, HAVING, SELECT (expressions and window functions), DISTINCT, ORDER BY, then LIMIT/TOP. It is the logical order, not necessarily how the optimizer physically runs it.

Q2. How do you delete duplicate rows but keep one?

Number the duplicates with `ROW_NUMBER() OVER (PARTITION BY key ORDER BY id)` in a CTE, then delete rows where the number is greater than 1.

Q3. How do you find users with 3 or more consecutive login days?

Gaps-and-islands: dedupe the dates, subtract `ROW_NUMBER()` from the date to get a group key, GROUP BY that key, and filter `HAVING COUNT(*) >= 3`.

Q4. How do you get the top N rows per group?

Rank rows with `ROW_NUMBER` (or `DENSE_RANK` for ties) partitioned by the group, in a subquery or CTE, then filter `rank <= N`.

Q5. OFFSET pagination vs keyset pagination?

`OFFSET n` still reads and discards n rows, so deep pages get slow. Keyset (seek) pagination uses `WHERE id > last_seen_id ORDER BY id LIMIT n` and stays fast with an index.

Q6. Find the second highest salary without window functions.

`SELECT MAX(salary) FROM employees WHERE salary < (SELECT MAX(salary) FROM employees);` It returns NULL when no second value exists.

16 Bonus: 10 Must-Practise SQL Problems

These show up again and again in interviews. Try to write each one yourself before reading the solution.

Problem 1. Second highest salary

SQL | Solution 1

```sql
SELECT MAX(salary) FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

Problem 2. Nth highest salary (N = 3)

SQL | Solution 2

```sql
SELECT DISTINCT salary FROM (
  SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS r
  FROM employees) t
WHERE r = 3;
```

Problem 3. Find duplicate emails

SQL | Solution 3

```sql
SELECT email, COUNT(*) FROM users
GROUP BY email HAVING COUNT(*) > 1;
```

Problem 4. Employees who earn more than their manager

SQL | Solution 4

```sql
SELECT e.name FROM employees e
JOIN employees m ON e.manager_id = m.emp_id
WHERE e.salary > m.salary;
```

Problem 5. Customers who never ordered

SQL | Solution 5

```sql
SELECT c.name FROM customers c
LEFT JOIN orders o ON o.customer_id = c.id
WHERE o.id IS NULL;
```

Problem 6. Highest paid employee in each department

SQL | Solution 6

```sql
SELECT * FROM employees e
WHERE salary = (SELECT MAX(salary) FROM employees
                WHERE dept_id = e.dept_id);
```

Problem 7. Month-over-month revenue change

SQL | Solution 7

```sql
SELECT month, revenue,
       revenue - LAG(revenue) OVER (ORDER BY month) AS change
FROM monthly_sales;
```

Problem 8. Running total of sales

SQL | Solution 8

```sql
SELECT order_date, amount,
       SUM(amount) OVER (ORDER BY order_date) AS running_total
FROM orders;
```

Problem 9. Swap 'M' and 'F' in one UPDATE

SQL | Solution 9

```sql
UPDATE person
SET gender = CASE gender WHEN 'M' THEN 'F' WHEN 'F' THEN 'M' END;
```

Problem 10. Median salary (PostgreSQL)

SQL | Solution 10

```sql
SELECT PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY salary)
FROM employees;
```

17 Cheat Sheet and Interview Strategy

LOGICAL EXECUTION ORDER

1	2	3	4	5	6	7	8
FROM / JOIN	WHERE	GROUP BY	HAVING	SELECT / windows	DISTINCT	ORDER BY	LIMIT / TOP

JOIN QUICK GUIDE

Join	Returns
INNER	Only rows matching in both tables
LEFT	All left rows + matches (NULL if none)
RIGHT	All right rows + matches (NULL if none)
FULL OUTER	All rows from both tables
CROSS	Every combination (m x n rows)
SELF	Table joined to itself via aliases

GOLDEN RULES

- Never use `= NULL`. Use `IS NULL`.
- Prefer `NOT EXISTS` over `NOT IN` when NULLs are possible.
- Prefer `UNION ALL` over `UNION` when duplicates are fine.
- Do not wrap indexed columns in functions inside WHERE.
- Select only the columns you need; avoid `SELECT *` in production.
- Use a window function when you need detail rows and an aggregate together.
- Test every UPDATE and DELETE with a SELECT first, inside a transaction.

HOW TO ANSWER IN AN INTERVIEW

1. **Clarify**: ask about ties, NULLs, duplicates and the database engine.
2. **Think aloud**: describe your approach before writing SQL.
3. **Build step by step**: start with the joins, add filters, then aggregation.
4. **Check edge cases**: empty groups, NULLs, ties, one-row tables.
5. **Discuss performance**: mention indexes and the execution plan.

A 30-DAY PRACTICE PLAN

Week	Focus	Daily goal
1	SELECT, WHERE, ORDER BY, GROUP BY, HAVING	5 easy problems
2	Joins, subqueries, set operations, CASE	4 easy and medium problems
3	CTEs, window functions, date and string functions	3 medium problems
4	Indexes, transactions, design, mock interviews	2 hard problems and 1 concept review

Found this useful?

Save this guide, share it with someone learning SQL, and practise one problem every day.